SPEC-DRIVEN DEVELOPMENT · TUTORIAL

คู่มือ: สร้างแอปวิเคราะห์ Sentiment ด้วย SDD + OpenCode

`/spec` → `/plan-spec` → `/tasks` → `/implement`

คู่มือนี้พาใช้ชุดคำสั่ง SDD ที่ตั้งไว้ใน repo (`/spec` → `/plan-spec` → `/tasks` → `/implement`) สร้างแอป command line ที่รับข้อความภาษาไทยหรืออังกฤษ แล้วให้ LLM ตอบว่าเป็น **positive**, **negative** หรือ **neutral**

หลักการเดิม: **AI เขียนเอกสาร → คุณอ่านและแก้ → ค่อยไปขั้นถัดไป** อย่าข้ามการตรวจ

สิ่งที่ต้องมีก่อนเริ่ม

รายการ	ตรวจอย่างไร
ไฟล์ SDD ครบ	มี `AGENTS.md` , `.opencode/agents/spec-writer.md` , `.opencode/commands/` 4 ไฟล์, โฟลเดอร์ `specs/`
Python environment	`pip install -r requirements.txt` ผ่าน (ใช้ `.venv/` ที่มีอยู่ได้)
API key	`.env` ที่รากโปรเจกต์มี `OPENROUTER_API_KEY=...`
OpenCode เชื่อมโมเดลแล้ว	เปิด `opencode` ได้ และเคย `/connect` แล้ว

เปิด OpenCode **ที่รากของ repo** (ไม่ใช่ใน `example/`):

```
cd vibe_coding_tutorial
opencode
```

พิมพ์ `/` ต้องเห็น `spec` , `plan-spec` , `tasks` , `implement` และกด Tab ต้องเห็น agent `spec-writer`

> **เรื่องอาร์กิวเมนต์:** `$1` คือคำแรก `$2` คือคำถัดไป ข้อความยาวที่มีช่องว่างต้อง **ครอบด้วยเครื่องหมายคำพูด** ไม่เช่นนั้น `$2` จะได้แค่คำเดียว

1 `/spec` เขียนสเปก (อะไร / เพื่อใคร / ทำไม)

ชื่อฟีเจอร์ในคู่มือนี้คือ `sentiment` พิมพ์:

```
/spec sentiment "โปรแกรม command line ให้ผู้ใช้พิมพ์ข้อความภาษาไทยหรืออังกฤษ แล้วบอกว่าข้อความนั้นมี sentiment เป็น positive, negative หรือ neutral พร้อมเหตุผลสั้น ๆ โดยใช้ LLM วิเคราะห์"
```

AI อาจถามกลับ — เตรียมคำตอบไว้

คำถามที่มักเจอ	คำตอบที่แนะนำสำหรับแบบฝึกนี้
รับข้อความทีละข้อหรือหลายข้อ?	ทีละข้อ พิมพ์ต่อได้เรื่อย ๆ จนพิมพ์ `exit`
ต้องมีค่าความมั่นใจ (confidence) ไหม?	ไม่ต้อง เอาแค่ label + เหตุผล 1 ประโยค
ข้อความผสมสองภาษาได้ไหม?	ได้
ข้อความว่างทำอย่างไร?	แจ้งเตือนและให้พิมพ์ใหม่ ไม่เรียก LLM
ถ้าเรียก LLM ไม่สำเร็จ?	แสดงข้อความ error ที่อ่านเข้าใจ โปรแกรมไม่ล่ม
เหตุผลตอบภาษาอะไร?	ภาษาเดียวกับข้อความที่ป้อน

คุณทำ: ตรวจ `specs/sentiment/spec.md`

เปิดไฟล์แล้วเช็กทีละข้อ:

- [ ] **ไม่มีชื่อเทคโนโลยี** (Python, CrewAI, OpenRouter ฯลฯ) — ถ้ามี สั่งให้ลบ เพราะเรื่อง "อย่างไร" อยู่ใน plan
- [ ] label มี **3 ค่าตายตัว** สะกดตรงกันทั้งไฟล์: `positive`, `negative`, `neutral`
- [ ] ทุก user story มีเกณฑ์ยอมรับแบบ Given/When/Then อย่างน้อย 1 ข้อ
- [ ] มีกรณีขอบ: ข้อความว่าง, ข้อความยาวมาก, LLM ตอบผิดรูปแบบ, เครือข่ายล่ม
- [ ] "นอกขอบเขต" ระบุชัด เช่น ไม่มีหน้าเว็บ, ไม่วิเคราะห์ไฟล์, ไม่เก็บประวัติ, ไม่ตรวจจับอารมณ์ละเอียด (โกรธ/เศร้า)

ตัวอย่างเกณฑ์ยอมรับที่ดี (ถ้า AI เขียนไม่ชัดขนาดนี้ ให้สั่งแก้):

```
- Given ผู้ใช้พิมพ์ "อาหารร้านนี้อร่อยมาก บริการดีสุด ๆ"
  When ระบบวิเคราะห์เสร็จ
  Then แสดงผลเป็น positive พร้อมเหตุผลภาษาไทย 1 ประโยค

- Given ผู้ใช้พิมพ์ "The package arrived on Tuesday."
  When ระบบวิเคราะห์เสร็จ
  Then แสดงผลเป็น neutral

- Given ผู้ใช้กด Enter โดยไม่พิมพ์อะไร
  When ระบบได้รับข้อความว่าง
  Then แสดง "กรุณาพิมพ์ข้อความ" และไม่เรียกตัววิเคราะห์

- Given ตัววิเคราะห์ตอบค่าที่ไม่ใช่ 1 ใน 3 label
  When ระบบอ่านผล
  Then แสดงข้อความแจ้งว่าวิเคราะห์ไม่สำเร็จ และโปรแกรมทำงานต่อได้
```

สั่งแก้ได้ตรง ๆ ในแชท เช่น `แก้เกณฑ์ยอมรับข้อ 3 ให้ระบุข้อความเตือนว่า "กรุณาพิมพ์ข้อความ"` ทำซ้ำจนพอใจ แล้ว commit:

```
git add specs/sentiment/spec.md && git commit -m "spec: sentiment"
```

2 `/plan-spec` วางแผนเทคนิค (อย่างไร)

ตอนนี้ค่อยบอกเทคโนโลยี ให้สอดคล้องกับที่ repo ใช้อยู่แล้ว:

```
/plan-spec sentiment "Python 3, ใช้ crewai LLM ผ่าน OpenRouter โมเดล openrouter/qwen/qwen3.8-27b base URL https://openrouter.ai/api/v1 อ่าน OPENROUTER_API_KEY จาก .env ด้วย python-dotenv, โค้ดอยู่ในโฟลเดอร์ sentiment/ แยกจาก example/, ทดสอบด้วย pytest โดย mock การเรียก LLM ทั้งหมด ห้ามเรียก API จริงในเทสต์"
```

คุณทำ: ตรวจ `specs/sentiment/plan.md`

- [ ] **ไม่มีของเกินที่สั่ง** — ฐานข้อมูล, FastAPI/Streamlit, Docker, cache ถ้ามีให้สั่งตัด
- [ ] **แยกส่วนที่ทดสอบได้ออกจาก LLM** โครงที่ควรเห็นประมาณนี้:

`sentiment/ ├── __init__.py ├── analyzer.py # สร้าง prompt, เรียก LLM, แปลงคำตอบเป็น label ├── parser.py # ตรวจ/แยก label + เหตุผลจากข้อความที่ LLM ตอบ (ไม่เรียก LLM) └── cli.py # วนรับข้อความ, จัดการข้อความว่าง/exit, แสดงผล tests/ ├── test_parser.py ├── test_analyzer.py # ใช้ fake LLM └── test_cli.py`

- [ ] **รูปแบบคำตอบของ LLM ถูกกำหนดชัด** เช่น ให้ตอบ JSON `{"label": "...", "reason": "..."}` และระบุว่าถ้า parse ไม่ได้หรือ label ไม่อยู่ใน 3 ค่าจะทำอย่างไร
- [ ] prompt สั่งให้รองรับไทย/อังกฤษ และให้เหตุผลเป็นภาษาเดียวกับข้อความ
- [ ] ระบุว่าต้องเพิ่ม `pytest` ใน `requirements.txt` (ตอนนี้ยังไม่มี)
- [ ] ทุกการตัดสินใจอ้างถึง user story ใน spec (เช่น "parser.py — รองรับ story 3")
- [ ] หัวข้อความเสี่ยงพูดถึง: LLM ตอบไม่ตรงรูปแบบ, ค่าใช้จ่าย/latency, ผลไม่คงที่ (ควรตั้ง `temperature` ต่ำ)

```
git add specs/sentiment/plan.md && git commit -m "plan: sentiment"
```

3 `/tasks` แตกเป็นงานย่อย

```
/tasks sentiment
```

คุณทำ: ตรวจ `specs/sentiment/tasks.md`

ควรได้ราว 6–10 ข้อ แต่ละข้อเล็กพอทำจบในรอบเดียว และมีเทสต์กำกับ ตัวอย่างลำดับที่ดี:

```
- [ ] 1. เพิ่ม pytest ใน requirements.txt, สร้าง sentiment/__init__.py และ tests/
      เทสต์: `pytest` รันได้ (0 tests ก็ผ่าน)
- [ ] 2. parser.py: แปลง JSON ที่ถูกต้องเป็น (label, reason)
      เทสต์: test_parser.py กรณี positive/negative/neutral
- [ ] 3. parser.py: จัดการ JSON เสีย, label แปลก, ตัวพิมพ์ใหญ่ ("Positive")
      เทสต์: test_parser.py กรณีผิดรูปแบบ → raise/คืนค่า error ตาม plan
- [ ] 4. analyzer.py: สร้าง prompt ที่ใส่ข้อความผู้ใช้และกำหนดรูปแบบคำตอบ
      เทสต์: prompt มีข้อความผู้ใช้และระบุ 3 label
- [ ] 5. analyzer.py: เรียก LLM (inject ได้) แล้วส่งต่อให้ parser
      เทสต์: fake LLM คืน JSON → ได้ label ถูก; fake LLM โยน error → จัดการตาม plan
- [ ] 6. cli.py: วนรับข้อความ, ข้อความว่างเตือนและไม่เรียก analyzer, `exit` ออก
      เทสต์: test_cli.py ป้อน input จำลอง ตรวจ output
- [ ] 7. cli.py: เชื่อม analyzer จริง (โหลด .env, สร้าง LLM จาก OpenRouter)
      เทสต์: ทดสอบด้วยมือตามเกณฑ์ยอมรับใน spec.md
```

สิ่งที่ต้องแก้ถ้าเจอ:

- ข้อไหนไม่มีบรรทัด "เทสต์" → สั่งให้เติม
- ข้อไหนใหญ่เกิน (เช่น "ทำ analyzer ทั้งหมด") → สั่งให้แตกย่อย
- มีข้อที่ไม่ได้มาจาก spec/plan → สั่งให้ลบ

```
git add specs/sentiment/tasks.md && git commit -m "tasks: sentiment"
```

4 `/implement` ลงมือทีละข้อ

```
/implement sentiment
```

คำสั่งนี้ให้ agent `build` ทำ **ข้อแรกที่ยังไม่ติ๊กเพียงข้อเดียว**: เขียนเทสต์ → เขียนโค้ด → รันเทสต์ → ติ๊ก `[x]` → หยุด

คุณทำ: ทุกครั้งหลังจบ 1 ข้อ

1. อ่านรายงานและ diff (`git diff`) — แก้เฉพาะไฟล์ที่ข้อนั้นระบุหรือไม่
2. รันเทสต์เองให้เห็นกับตา:

`bash python -m pytest -q`

3. ตรวจว่าเทสต์ **ไม่เรียก API จริง** (ปิดเน็ตแล้วรันก็ควรผ่าน)
4. ผ่านแล้ว commit แล้วสั่งข้อถัดไป:

`bash git add -A && git commit -m "sentiment: task 1"`

`/implement sentiment`

วนจนติ๊กครบทุกข้อ

> ถ้า AI เจอว่า spec ขัดกันหรือไม่ครบ มันควรหยุดถามตามกติกาใน `AGENTS.md` ให้กลับไปแก้ `spec.md` (และ `plan.md` / `tasks.md` ที่เกี่ยวข้อง) ก่อน อย่าให้มันเดาในโค้ด

5 ปิดงาน: ทดสอบกับของจริงตามเกณฑ์ยอมรับ

รันแอป (คำสั่งจริงดูจาก `plan.md` ปกติจะเป็นแบบนี้):

```
python -m sentiment.cli
```

ป้อนตัวอย่างเหล่านี้ แล้วเทียบกับเกณฑ์ยอมรับใน `spec.md` ทีละข้อ:

ข้อความ	ควรได้
อาหารร้านนี้อร่อยมาก บริการดีสุด ๆ	positive
รอคิวสองชั่วโมง พนักงานไม่สนใจเลย	negative
พรุ่งนี้ประชุมตอนบ่ายสองโมง	neutral
I absolutely love this phone!	positive
The update broke everything. Terrible.	negative
The package arrived on Tuesday.	neutral
หนังสนุกนะ but the ending was disappointing	ยอมรับได้ทั้ง negative/neutral — ดูว่าเหตุผลสมเหตุสมผล
(กด Enter เฉย ๆ)	ข้อความเตือน ไม่เรียก LLM
`exit`	ออกจากโปรแกรม

ถ้าผลต่างจากสเปก **อย่าแก้โค้ดตรง ๆ** ให้วนกลับ:

1. แก้ `spec.md` (หรือเพิ่มเกณฑ์ยอมรับใหม่)
2. `/tasks sentiment` เพื่อเพิ่มงานที่ขาด
3. `/implement sentiment` จนครบ

โครงสร้างเมื่อจบ

```
vibe_coding_tutorial/
├── AGENTS.md
├── .opencode/
│   ├── agents/spec-writer.md
│   └── commands/spec.md, plan-spec.md, tasks.md, implement.md
├── specs/sentiment/spec.md, plan.md, tasks.md
├── sentiment/          # โค้ดที่ AI สร้าง
├── tests/              # เทสต์ที่ AI สร้าง
├── example/crew_ai.py  # ของเดิม ไม่ถูกแตะ
└── requirements.txt    # มี pytest เพิ่ม
```

ติดปัญหา?

อาการ	วิธีแก้
spec ได้แค่คำเดียวจากคำอธิบาย	ลืมครอบ `$2` ด้วยเครื่องหมายคำพูด
`spec-writer` เขียน spec.md ไม่ได้	ตรวจลำดับกฎใน `spec-writer.md` : deny `*` ก่อน allow `specs/**`
AI อยากอ่าน `.env` แต่ถูกปฏิเสธ	ถูกต้องแล้ว — `opencode.jsonc` ห้ามไว้ โค้ดอ่านคีย์ตอนรันผ่าน `load_dotenv()` เอง
`KeyError: 'OPENROUTER_API_KEY'`	รันจากรากโปรเจกต์ และตรวจว่า `.env` มีคีย์
เทสต์ช้า/เสียเงิน	เทสต์เรียก LLM จริง — สั่งให้เปลี่ยนเป็น fake LLM ตามที่ plan กำหนด
LLM ตอบ "Positive." หรือมีข้อความเกิน	ให้ parser normalize (ตัวพิมพ์เล็ก, ตัดช่องว่าง/จุด) และเพิ่มเทสต์กรณีนี้
ผลเปลี่ยนไปมาเมื่อรันซ้ำ	ตั้ง `temperature=0` ใน plan แล้ว `/implement` ใหม่
`/implement` ทำหลายข้อรวด	พิมพ์ย้ำ "ทำข้อเดียวแล้วหยุด" แล้วตรวจ diff ก่อน commit